APIShield: A Local Lab for Attacking, Detecting and Defending Against Common API Security Risks

Abirami P C
24i302@psgtech.ac.in
Department of Information Technology
PSG College of Technology, Coimbatore, India

***Abstract*—Most apps on my phone get their data from a server through an API. If that API is written carelessly an attacker needs no clever trick. Often they only have to ask for somebody else's data. APIShield is a small lab I built on my own laptop to study this. One FastAPI server holds six deliberately weak behaviours taken from the OWASP API Security Top 10 (2023). Each one has a detector and a switch that turns its defense on or off while the server keeps running. For every weakness a short Python script sends the same attack twice, first with the defense off and then with it on and afterwards reads the security log to check that the attack was noticed. My saved runs cover five categories: broken object level authorization, broken authentication, broken object property level authorization, unrestricted resource consumption and broken function level authorization. In all five the attack worked with the defense off and failed with it on and a matching log entry existed each time. The sixth category (improper inventory management) is built and tested but has no saved run. The paper also surveys recent literature, walks through the code and the data flow diagrams in plain words and states clearly that the results come from single runs on one machine.**

***Index Terms*—API security, OWASP API Top 10, broken access control, JWT, rate limiting, security event logging, FastAPI**

I. Introduction

Think about opening a food delivery app or checking a bank balance on your phone. The app does not hold your data itself. It sends small requests to a server and the server answers. The list of addresses a server answers to is its API (application programming interface). A request like `GET /items/3` just means "give me item number 3".

That sounds harmless but the server has to settle one question every single time: who is asking and may this person see this particular thing? If the programmer skips that check even once, any logged-in user can read other people's data by changing a number in the address. No hacking tool is needed. The OWASP Foundation publishes a list of the ten most common API mistakes and its 2023 edition is where my project starts [1].

The trend is going the wrong way. Akamai's 2026 State of the Internet report, as summarised by Infosecurity Magazine, puts the average number of API attacks per organization at 121 in 2024 and 258 in 2025, a rise of 113% [2]. Traceable AI found that 57% of surveyed organizations had an API related data breach within two years and only 21% felt they could detect attacks at the API layer well [3]. Both come from security vendors so I do not lean on the exact figures, but they agree with each other on direction.

I built this project because I learn better by building. Most tutorials show an attack that works and stop there. Most security products show a block and never show what would have happened without it. I wanted to see both on the same endpoint under the same conditions with the evidence written down. That became APIShield.

What this paper contributes

- A small FastAPI lab with six selected OWASP API risks, each with a runtime switch for its defense.
- A repeatable routine I call *attack, detect, defend, verify*. Every attack runs twice and the result is checked against a structured security log and not only against the HTTP status code.
- A walk through the real source code, with data flow diagrams from Level 0 down to Level 4 and a full list of the thresholds the code uses.
- A literature survey of standards, industry reports and research papers and a plain statement of where this project fits and where it does not.

The paper is organised as follows. Section II covers the basic ideas. Section III is the literature survey. Sections IV to VII describe the problem, the design, the code and the attack modules. Section VIII has the data flow diagrams. Sections IX and X cover the experiments and results and Sections XI to XIII give limitations, future work and the conclusion.

II. Background for Beginners

A. APIs, requests and status codes

A client, which can be a mobile app, a web page or a Python script, sends an HTTP request to an endpoint. The server replies with data and a three digit status code. Five codes matter here: 200 (success), 401 (not logged in or a bad token), 403 (logged in but not allowed), 404 (not found) and 429 (too many requests).

B. Authentication versus authorization

People mix these two up all the time. Authentication answers "who are you?" and authorization answers "what may

you do?". A college gate is a good picture. The guard checks your ID card at the gate, which is authentication, but that card does not get you into the exam cell, which is authorization. Many API bugs have nothing to do with weak logins. The user is logged in properly and the server just never asks the second question.

C. Passwords and tokens

APIShield never keeps a password as typed. It keeps a bcrypt hash, a one way scrambled version that is slow to compute on purpose, so guessing passwords from a stolen database costs a lot of time [4]. After a correct login the server returns a JSON Web Token (JWT), a short signed string that the client attaches to later requests as proof of who it is [5]. The signature stops the user from editing the token. The IETF also publishes advice on using JWTs safely [6].

D. Roles

Every user has a role, either `user` or `admin`. Giving different roles different powers is called role based access control [7].

E. Where the API lives in APIShield

The API here is a FastAPI server listening on `127.0.0.1:8000` so only my own computer can reach it. Table I lists its endpoints. Two of them are the main targets: `/items/{id}`, which returns a stored item and `/admin/events`, which returns the security log. The database holds three synthetic users (`alice` and `bob` with role `user` plus one `admin`) and four items, two owned by Alice and two by Bob.

F. The six OWASP risks covered

OWASP's 2023 list has ten items [1]. I implemented six, shown in Table II. The other four are not covered and the paper does not pretend otherwise.

TABLE I
ENDPOINTS OF THE APISHIELD SERVER

Method	Path	Who may call	Purpose
POST	`/auth/register`	Anyone	Create an account (role is always user)
POST	`/auth/login`	Anyone	Check password, return a JWT
GET	`/items/{id}`	Logged in	Read one item
GET	`/admin/events`	Admin	List security events
GET, PATCH	`/admin/defenses`	Admin	Read or change the six defense flags
GET	`/admin/lab-mode`	Admin	Read lab mode (read only)
GET	`/admin/inventory`	Admin	List known routes
GET	`/api/v0/items/{id}`	Logged in	Old route, reachable only in lab mode
GET	`/health`	Anyone	Check that the server runs

TABLE II
OWASP API SECURITY TOP 10 (2023) ITEMS COVERED

ID	Name (short)	Plain meaning
API1	BOLA	Reading or changing another user's object by changing its ID
API2	Broken authentication	Unlimited password guessing
API3	BOPLA	Seeing fields of an object that should stay hidden
API4	Resource consumption	Sending so many requests that the service suffers
API5	BFLA	A normal user calling an admin only function
API9	Improper inventory	Old, forgotten API versions still reachable

III. LITERATURE SURVEY

A. How I searched

I searched online for recent surveys, conference papers, standards and industry reports on API security and REST API testing. My search terms included REST API security testing, broken object level authorization, broken access control fuzzing, excessive data exposure, mass assignment and state of API security. I kept sources that were widely used standards, peer reviewed or archived research or large industry reports from 2019 to 2026. For each one I read the abstract and the method and results parts. I did not manage to read every paper in full so a few entries below rest on abstracts and summaries and I say so where it matters.

B. What I found in short

Four themes kept coming back. Attacks on APIs are rising fast and most of them are ordinary. The worst flaws are authorization flaws, where the server knows who you are but does not check what you may touch. Automated tools find crashes far more easily than access control mistakes. And many papers end by asking for better ways to judge whether a tool really found a security problem. The next sections go through these one by one.

C. Standards and guidance

Almost every paper I read uses the OWASP API Security Top 10 as its common vocabulary. The 2023 edition puts broken object level authorization at number one with broken

authentication, object property level authorization and unrestricted resource consumption right behind it [1]. For tokens and passwords I used RFC 7519 for JWT [5], RFC 8725 for JWT best practice [6] and the original bcrypt paper [4]. Roles follow the classic RBAC models [7]. For the login lockout I checked NIST's digital identity guidance, which tells servers to limit consecutive failed attempts on one account and sets 100 as the upper bound [8]. The 429 status code comes from RFC 6585 [9].

D. What the industry reports say

Three reports gave me a picture of the current situation. Akamai says daily API attacks per organization more than doubled between 2024 and 2025 [2]. Traceable's 2025 report names detection as the weak point since only about one organization in five rated its API attack detection as strong [3]. Salt Security's Q1 2025 report, built on 206 survey respondents and customer data, says 99% of respondents had API security problems in the previous twelve months [10]. All three are written by companies that sell API security products and two are surveys. I use them for direction only and not as exact measurements.

E. Research on testing REST APIs

Golmohammadi, Zhang and Arcuri reviewed 92 papers on testing RESTful APIs. They found steady growth in interest since 2017 and many open problems [11]. Zhang and Arcuri ran fuzzing tools on real services and listed where they still fail [12]. Kim et al. compared ten REST test generators on twenty open source services [13]. RESTTESTGEN builds tests from the API description and tries normal as well as invalid inputs [14]. A newer survey by Tanveer et al. focuses on vulnerability detection for REST APIs and offers a taxonomy that includes authentication and authorization problems [15].

RESTler, from Microsoft Research, is a well known stateful fuzzer that chains requests together using an API description [16]. The same group later added security rules such as "a deleted resource must not be reachable" and found real bugs in Azure and Office 365 services [17]. Du et al. presented vulnerability oriented testing for RESTful APIs at USENIX Security 2024 [18]. One limit shows up across these tools. They are good at finding crashes and server errors but a security hole that returns a perfectly normal 200 is much harder for them to spot.

F. Research on authorization flaws

That limit is the reason authorization bugs are hard. When Alice reads Bob's item nothing crashes and the response looks fine. The tool has to know that Alice should not have received it. The BACFuzz authors call this the oracle problem and they also point at the difficulty of building requests valid enough to pass input checks [19]. Their grey-box fuzzer found 16 of 17 known cases and reported 26 new access control flaws in 20 PHP web applications. Huang et al. went after BOLA with static analysis of database backed applications [20]. A Palo Alto Networks Unit 42 article describes BOLABuster, which uses a large language model to read an API description and generate BOLA tests and it reports using under one percent of the requests that ordinary fuzzers send [21].

TABLE III
TRENDS SEEN IN THE LITERATURE

Trend	What the sources say	Source
More attacks	Average API attacks per organization rose from 121 to 258 between 2024 and 2025	[2]
Weak detection	Only 21% of surveyed organizations rated their API attack detection as strong	[3]
Wide exposure	99% of respondents reported an API security problem in a year	[10]
Authorization first	BOLA is ranked first in the 2023 list	[1]
Oracle problem	Tools struggle to tell a harmless 200 from a leak	[19]
Fewer requests	LLM guided testing reports using under 1% of the requests of fuzzers	[21]
Research growth	92 studies on REST API testing reviewed, interest rising since 2017	[11]

TABLE IV
RELATED WORK COMPARED WITH APISHIELD

Work	Focus and what it leaves out	What APIShield does
OWASP Top 10 [1]	Names and ranks risks, nothing runnable	Six risks as runnable experiments
RESTler and checkers [16], [17]	Fuzzes cloud APIs to find bugs, no defense off and on comparison	Same endpoint, defense toggled, result logged
Test generator studies [11]–[13]	Coverage and fault finding, few security oracles	A security event is the oracle
BACFuzz [19]	Finds BOLA and BFLA in PHP apps, does not model the fix	Models the fix, shows before and after
BOLA static analysis [20]	Static analysis, no runtime attack	Live attack with live detection
LLM assisted BOLA tests [21]	Generates test cases, detection only	Detect, defend and verify
EDEFuzz, mass assignment [22], [23]	One property problem each	Property read flaw inside a multi-risk lab
Industry reports [2], [3], [10]	Trends and surveys, not technical evidence	Used as motivation only

G. Research on property level problems

For API3, Corradini et al. built a black-box tester for mass assignment, where a client sends fields it should not be allowed to set [22]. Pan et al. built EDEFuzz to find excessive data exposure, where a server returns more fields than the client needs [23]. Between them they cover the write side and the read side of the property problem. APIShield's BOPLA module is a small example of the read side.

H. Current trend in one view

Table III lists the main trends with the evidence behind each. They are the reason APIShield concentrates on authorization and on checking evidence.

I. Summary and gap

Table IV sets the main related work next to APIShield.

Here is the pattern I see. Standards describe the risks. Fuzzers and analysers try to *find* holes in someone else's API.

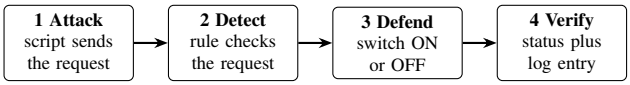


Fig. 1. The attack, detect, defend, verify loop. It runs once with the defense off and once with it on.

Industry products try to *block* attacks in production. In the sources I read I could not find a small repeatable setup where a student takes one vulnerability, attacks it, watches it get detected, turns on the fix, attacks again and reads the evidence, all on a laptop. APIShield is meant for that gap. It is a teaching and experiment platform. It does not replace the research tools above and it does not claim to find unknown bugs.

IV. Problem Statement and Objectives

Problem. Developers and students tend to meet an API vulnerability either as a story ("this can happen") or as a product feature ("we block this"). It is hard to follow one vulnerability from start to finish under controlled conditions, including whether the detector really fired and whether the defense really changed the result.

Objectives.

1) Build one local API that can behave in a vulnerable way, safely, for six OWASP categories.
2) Give each category a detector that writes a structured security event.
3) Give each category a runtime switch that turns its defense on or off without a restart.
4) Write one attack script per category that attacks with the defense off then on and checks the event.
5) Keep everything local, synthetic and clearly marked so it cannot be pointed at a real system by accident.

V. System Design

A. Attack, detect, defend, verify

Every experiment follows the four steps in Fig. 1. For each risk the loop runs twice, once with the defense switch off and once with it on.

B. Architecture

Fig. 2 shows the parts. The React dashboard and the Python attack scripts both talk to one FastAPI server. Inside it a gateway middleware times every request, the route handlers do the normal work, the detectors and the defense manager make the security decisions and the important records go into a SQLite database. The server uses FastAPI with Uvicorn [24], SQLite through SQLAlchemy, bcrypt and JWT (HS256) for passwords and tokens, Pydantic for input checks, HTTPX in the attack scripts and React with Vite for the dashboard.

C. Safety rules built into the design

Some of this code is vulnerable on purpose so I put safety rules into the specification and into the code. The server binds only to `127.0.0.1`. Each attack script checks its target against a hard coded allow list (local host and port 8000) and refuses to run on anything else. All users and data are

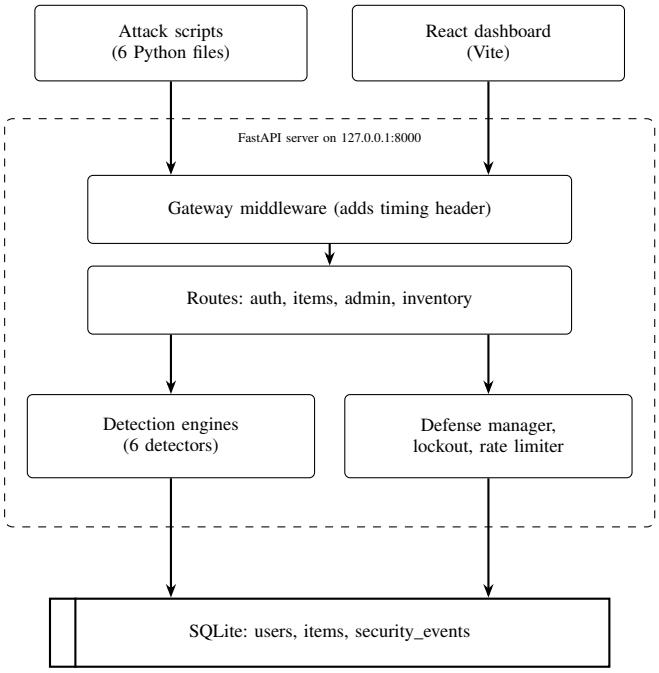


Fig. 2. High level architecture of APIShield.

synthetic. The JWT secret comes from the environment. If it is missing, still a placeholder or shorter than 32 characters, the server creates a random one-time key and prints a warning. Every vulnerable branch in the code has a comment starting with `VULNERABLE` that names the OWASP item.

The most important rule is *lab mode*. The server reads it from an environment variable at startup and nobody can switch it on over HTTP. When it is off, which is the default, all vulnerable branches stay closed and the server acts like a normal safe API. A vulnerability can be reached only when lab mode is on *and* the matching defense switch is off. My notes from the BOLA module say the same thing. With only a runtime toggle a normally started server could still show the weakness, so the startup opt-in became the first line of defense.

D. Threat model

The attacker I model is an ordinary logged-in user with a synthetic account. This person can send any request to the local server, knows how the API is laid out, can read their own token and cannot forge a new one without the signing secret. I left several things out of scope: attackers on the network (the server listens only locally), anyone who can read the computer's files, real credentials, race conditions between several server workers and attacks on the database file itself. The model is narrow on purpose because the lab is about mistakes in authorization logic and not about network attacks.

VI. Implementation and Code Analysis

I went through the source package file by file. The backend is about 1,750 lines of Python, the six detectors about 330 lines, the attack scripts about 2,600 lines, the tests about 4,600

lines and the dashboard about 500 lines. The top level folders are `backend`, `detection`, `attacks`, `tests`, `scripts` and `frontend`.

A. Authentication code

Registration takes only a username and a password. The request model rejects any extra field so a client cannot add `role=admin` to promote itself. Passwords are hashed with bcrypt at cost 12. A successful login returns a JWT signed with HS256. It holds the user ID, the issue time and an expiry 30 minutes later plus the username and role.

My progress notes list one early defect in this code. The first version of the config fell back to the string `change-me` when no signing secret was set, which means anyone who read the source could forge a token. I replaced it with the startup check described in Section V-C.

I like one design choice here. On every later request the server checks the signature and expiry, reads the user ID and then loads the user from the database. So the role used for decisions comes from the database and not from the token. If an admin is demoted the old token loses its admin power immediately.

B. The gateway middleware

The gateway is kept small. It starts a high resolution timer, lets the request through and adds an `X-Response-Time-Ms` header to the response. The experiments read this header to measure latency. Security decisions are not made here. They happen in the dependencies and route handlers.

C. Defense manager

The defense manager holds six true or false flags: `bola`, `auth_hardening`, `schema_validation`, `rate_limit`, `rbac` and `deprecated_route_protection`. All start as false. An admin reads them with `GET /admin/defenses` and changes them with `PATCH /admin/defenses`. The update model is strict and unknown names, non boolean values and empty bodies get status 422. The flags sit in memory. That makes them quick and easy to reset in tests but they go back to off whenever the server restarts.

D. Detectors and the security event log

Each detector is a short function. It first checks whether the request is legitimate, for example whether the user owns the object or is an admin. If so it returns nothing and no event is written. If not it writes a security event with the action `detected` when the defense is off and `blocked` when it is on. An event stores an ID, a UTC time, the user, the endpoint and method, the object involved, the OWASP category, a severity from LOW to CRITICAL, the rule name and the action. The event table also has a database level check that allows only those two action values on top of the same check in Python.

One thing took me a while to appreciate: the *fail closed* rule. If saving an event to the database fails, the BOLA, BOPLA, BFLA, inventory and rate limit detectors raise HTTP 500 and the request is refused. Denying a request is safer than allowing one that cannot be audited. The authentication detector is the one exception. It catches the failure and carries on so a failed login still gets its normal answer.

TABLE V
THRESHOLDS THAT EXIST IN THE CODE

Setting	Value	File
JWT lifetime	30 minutes	`config.py`
Shortest accepted JWT secret	32 characters	`config.py`
bcrypt cost	12	`auth/security.py`
Username length	3 to 50	`routes/auth.py`
Password length	8 to 72	`routes/auth.py`
Failed logins before lockout	5	`lockout.py`
Window for counting failures	300 s	`lockout.py`
Lockout duration	300 s	`lockout.py`
Requests allowed per user	20 per 60 s	`rate_limiter.py`
Events per page (admin)	1 to 100	`routes/admin.py`
Wrong passwords sent by attack	7 (cap 10)	`attacks/auth.py`
Burst size of attack	25 (cap 50)	`attacks/rate_limit.py`

E. Lockout and rate limiting

Both defenses are small classes that guard their data with a lock so they are thread safe. My early progress notes list the lockout as missing even though its `auth_hardening` switch already existed, so for a while the toggle changed nothing. The finished login tracker remembers failed attempts per username over a five minute window. After five failures the account is locked for five minutes and the server answers 429 with a `Retry-After` header. The rate limiter is a fixed window counter keyed by user ID and by default it allows 20 requests in 60 seconds. Old windows are removed on every check so memory use stays bounded.

F. Thresholds and limits

Several places in the code use a fixed number. I collected all of them from the source files so nothing is hidden. Table V lists the values that exist in the code and were used in the experiments. I did not change any of them for this paper.

Two details are worth noting. The lockout starts when the fifth failure is recorded. Attempts one to five therefore get 401 and the sixth gets 429, which matches the saved run. Also the rate limiter is a fixed window counter. The twenty-first request inside the same 60 seconds is refused and the count restarts only when the window ends. A comment in the code says “rolling” but the logic resets the counter so I call it fixed. A known weakness of a fixed window is that a burst straddling two windows can briefly pass up to twice the limit.

Rules with no number. The BOLA, BOPLA, BFLA and deprecated route detectors have no threshold in the code. They are yes or no rules about ownership, role, a query flag or a route name so one bad request is enough to raise an event.

TABLE VI
PROPOSED THRESHOLDS, NOT IMPLEMENTED

Area	Proposed rule
BOLA	5 or more requests for other users' items by one user within 60 s raises the severity from HIGH to CRITICAL
BOPLA	3 or more `include_internal` requests by a non-admin within 300 s raises an alert
BFLA	3 or more admin function calls by a non-admin within 60 s blocks that user for 5 minutes
Inventory	10 or more calls to a deprecated route within 1 hour raises an alert
Login endpoint	10 login requests per minute from one address in addition to the per username lockout
Lockout time	Doubles on each repeat, 5 then 10 then 20 minutes

That is simple and reliable for a lab. It also means they cannot notice a slow pattern, such as one user quietly trying many other people's items.

Proposed values (not in the code). For the places where the code has no number I picked some thresholds of my own, listed in Table VI. They are design suggestions based on my judgement. They are *not implemented*, no experiment used them and I have not measured whether they are good values. Treat them as a starting point for future work.

For comparison NIST allows up to 100 consecutive failed attempts on one account before throttling becomes mandatory [8] so my limit of 5 is far stricter. A strict number protects well against guessing but it also lets an attacker lock a real user out on purpose. NIST suggests delays and extra checks for this reason. RFC 6585 defines the 429 status code that both the rate limiter and the lockout return [9].

G. How the item endpoint decides

`GET /items/{id}` is the busiest part of the code so I traced it step by step. It runs the rate limit check, looks up the item (404 if missing), checks ownership and then checks whether the caller asked for internal fields. Outside lab mode a cross-user request always gets 403 and a non-admin asking for internal fields receives only the public fields. Inside lab mode the same situations call the detector and the matching defense flag chooses between a 403 and the vulnerable answer. One detail to be honest about: with the BOLA flag off the vulnerable answer returns only the public fields of the other user's item and not the internal notes. The leak is real but limited to those fields.

H. Attack scripts

All six scripts follow one pattern. Each validates the target URL, logs in as a synthetic user, runs the attack, logs in as admin to read the event log through the API, looks for the matching event and writes a JSON result file. The dual run helper turns the defense on through the admin API between the two runs. The React dashboard loads those JSON files and an admin can also use it to flip defense switches and browse events.

Two problems in my first BOPLA runner shaped how the scripts check themselves now. The result was only a text string so it could say success even when the detection check had failed. I added a `verification_passed` true or false field that code can read directly. I also made each script note the newest event ID before the attack and accept only a later event that matches on seven things: the ID, category, rule, endpoint, user, object and action. This stops an old log entry from being counted by accident. The dual run also restores the original defense setting in a `finally` block so a crash cannot leave a defense switched on.

TABLE VII
ATTACK MODULES, DETECTOR RULES AND EXPECTED BEHAVIOUR

ID	Attack in short	Rule name and defense switch	Defense OFF / ON
API1	Alice requests Bob's item 3	`bola_cross_tenant_access`; switch `bola`	200 / 403
API2	Repeated wrong passwords for one account	`auth_brute_force`; switch `auth_hardening`	401 only / 401 then 429
API3	Alice asks for internal fields of her own item	`bopla_excessive_data_exposure`; switch `schema_validation`	200 with hidden fields / 403
API4	Burst of 25 quick requests	`rate_limit_exceeded`; switch `rate_limit`	200 for all / 429
API5	Alice calls `GET /admin/events`	`bfla_unauthorized_function_access`; switch `rbac`	200 / 403
API9	Alice calls old route `/api/v0/items/{id}`	`deprecated_route_access`; switch `deprecated_route_protection`	200 / 403

```
GET /items/3     Authorization: Bearer <alice token>

Defense OFF  ->  200  {"id": 3, "title":
                 "Bob Financial Report", ...}
Defense ON   ->  403  {"detail": "Not authorized
                 to access this item"}
```

Fig. 3. The BOLA request as Alice sends it and the two answers.

I. Testing

The tests use an in-memory SQLite database that is rebuilt for every test so the development database stays untouched. They cover the safe baseline, the vulnerable behaviour, the defended behaviour and the fact that legitimate users such as owners and admins create no events. I ran the suite from the unzipped package on a fresh install and all 162 tests passed.

VII. THE SIX ATTACK MODULES

Table VII summarises the modules and the paragraphs after it explain each one in simple words. The attacker is always `alice`, an ordinary synthetic user, unless stated otherwise.

API1, BOLA. Alice owns items 1 and 2 and Bob owns items 3 and 4. Alice logs in correctly and then asks for item 3. Her token is valid so authentication passes and only the ownership check can stop her. The detector compares the item's owner with Alice's ID and writes an API1 event with severity HIGH.

API2, broken authentication. The script sends seven wrong passwords for `alice`. With the defense off every one gets 401 and the account is never protected. With the defense

on the first five get 401, the sixth triggers the lockout and the rest get 429.

API3, BOPLA. An item has two hidden fields, `internal_notes` and `is_confidential`. Alice adds `include_internal=true` to a request for her own item. Admins may see those fields and ordinary users may not. Alice is the owner here so this is a property problem and not an object problem. Adding the two columns also caught me out. SQLAlchemy's `create_all()` does nothing when a table already exists so my existing development database kept its old shape and the new fields were missing. I added a check that reads the table's columns and adds only the missing ones, so no data is deleted.

API4, resource consumption. The script fires a burst of 25 requests. A normal user rarely needs that many within a few seconds but a script can send them at once. With the defense on, requests past the limit get 429.

API5, BFLA. This one is about functions and not objects. `GET /admin/events` is for admins. Alice is logged in but not an admin. If the server only asks "are you logged in?" she gets through. My first plan was to loosen the shared `require_admin` check because that looked like the easy way to write this experiment. I dropped the plan when I saw it would also open `/admin/defenses` and `/admin/lab-mode` to ordinary users. The weakness now lives only inside the `/admin/events` handler and every other admin route keeps the strict check.

API9, improper inventory. The server has a pretend old version of the items endpoint under `/api/v0`. Outside lab mode it answers 404 as if it had been removed. In lab mode it still returns item data unless the protection switch is on. The admin endpoint `GET /admin/inventory` lists known routes including the deprecated one.

VIII. Data Flow Diagrams

A data flow diagram (DFD) shows how data moves through a system. I use these symbols: a plain rectangle is an external entity, a rounded box is a process and a box with a bar on its left is a data store. Arrows are data flows and each carries a label. The levels go from the whole system down to the point where the security decision is made. Because the paper has narrow columns, Levels 1 and 3 are each drawn as two connected diagrams.

A. Level 0

Fig. 4 shows APIShield as a single process. People and scripts send requests and credentials, the system sends responses back and it keeps its records in the data stores.

B. Level 1

Level 1 splits the system into five processes and five data stores. Process 1.0 handles login. Process 2.0 serves requests and does the normal authorization checks. Process 3.0 makes the security decision. Process 4.0 writes events. Process 5.0 lets the admin read events and change defense switches. The stores are D1 users, D2 items, D3 security events, D4 defense state and D5 counters (failed logins and request counts). Fig. 5 shows the user side and Fig. 6 shows the security and admin side.

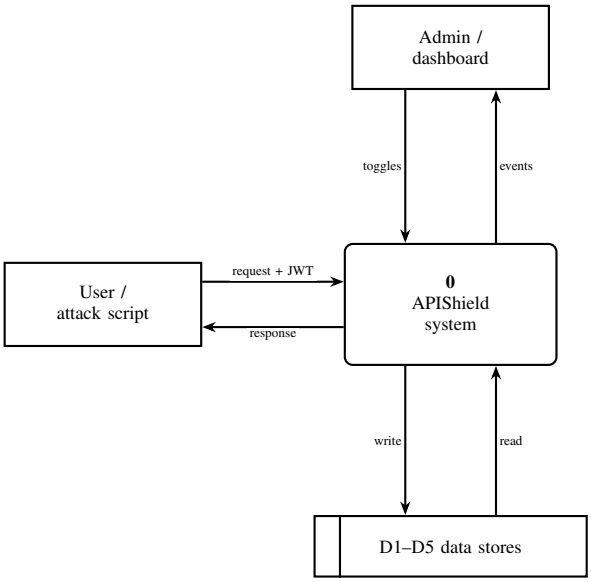


Fig. 4. DFD Level 0, context diagram.

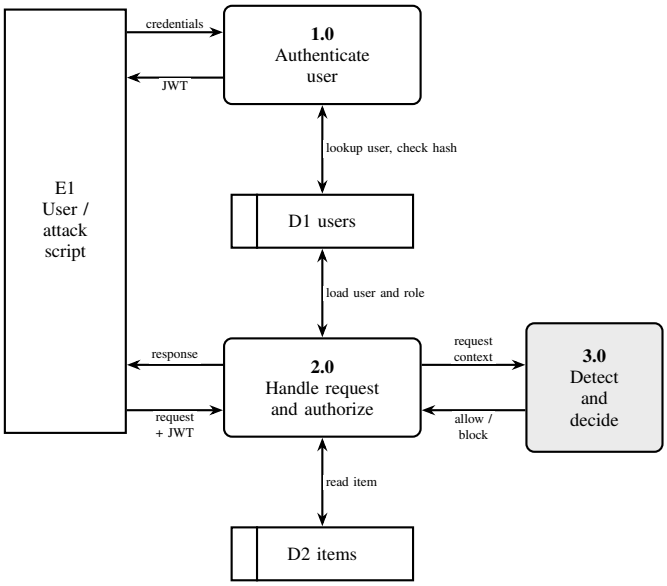


Fig. 5. DFD Level 1 (a), the user side: login (1.0) and request handling (2.0). Process 3.0 is expanded in Fig. 6.

C. Level 2: handling a request (process 2.0)

Fig. 7 opens process 2.0 and follows the order of the real code. The request first passes the gateway, which starts the timer. Next the JWT is verified and the user is loaded from D1. The rate limit counter in D5 is checked after that. Then the item is read from D2, ownership or role is checked and finally the property filter decides whether hidden fields may be included. Whenever one of these checks finds something

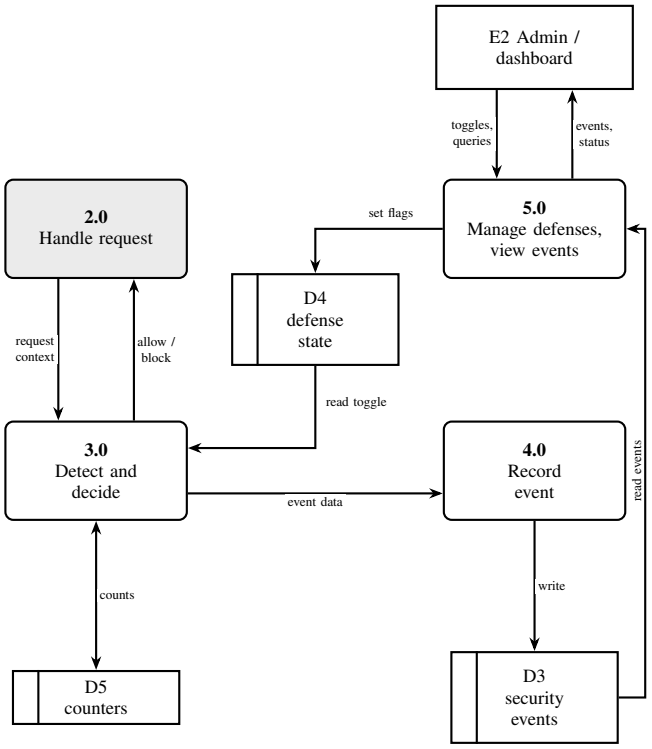


Fig. 6. DFD Level 1 (b), the security and admin side: detection (3.0), event recording (4.0) and defense management (5.0).

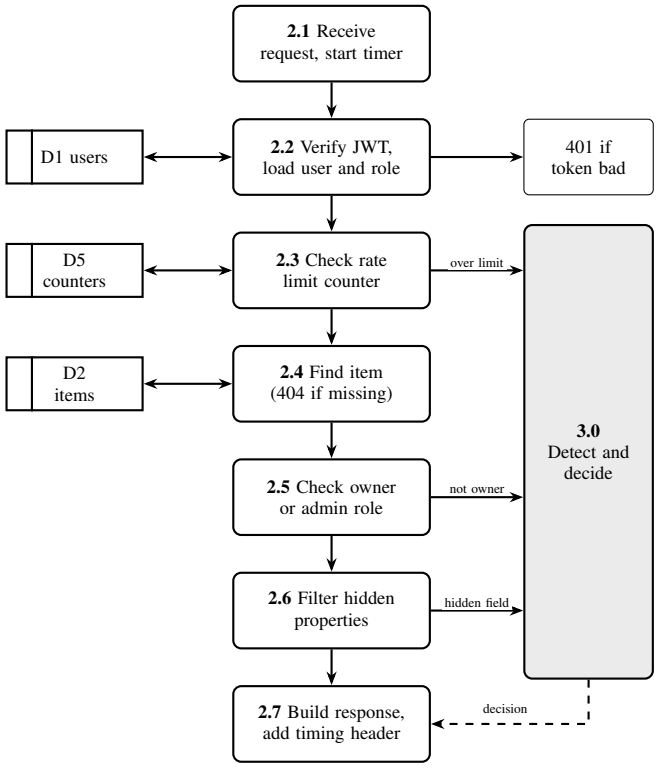


Fig. 7. DFD Level 2, process 2.0 in the order the code runs.

suspicious it passes the request context to process 3.0, which Level 3 explains in detail.

D. Level 3: detect and decide (process 3.0)

Level 3 is the core of APIShield and the part I spent the most time on. Process 3.0 breaks into eight steps. Figure 8 covers steps 3.1 to 3.4 where the request is checked and Fig. 9 covers steps 3.5 to 3.8 where the outcome is decided and returned.

1) **3.1 Lab mode gate.** With lab mode off nothing vulnerable is reachable. The request gets the safe answer (403 or 404) and no event is written.
2) **3.2 Read toggle.** The matching defense flag is read from D4.
3) **3.3 Select detector.** Depending on what is requested one of the six detectors is picked (API1, API2, API3, API4, API5 or API9).
4) **3.4 Evaluate rule.** The detector asks its own question. Does the user own the object? Is the user an admin? Was a hidden field requested? Is the counter over the limit? Were there too many failed logins? Is the route deprecated? Counters come from D5 and objects from D2.
5) **3.5 Choose action.** A legitimate request ends before this step with no event. A suspicious one gets the action `detected` if the flag is off and `blocked` if it is on.
6) **3.6 Write event.** The event is saved in D3 through process 4.0. If saving fails most detectors fail closed with HTTP 500.
7) **3.7 Enforce.** With the flag on the request is refused (403 or 429). With the flag off it continues and the vulnerable answer comes back.
8) **3.8 Respond.** The result goes back through the gateway, which adds the timing header.

I went one level further, to Level 4 for step 3.4, because that is where the actual rules and numbers sit. Below that the diagram would turn into code.

E. Level 4: evaluate rule (process 3.4)

Fig. 10 opens step 3.4. Here every detector asks its own question and here the thresholds from Section VI-F are used. Each question has a yes or no answer. A *yes* means the request is suspicious and moves on to step 3.5. A *no* means it is legitimate, nothing is logged and the request carries on. Four of the six questions contain no number at all because they are plain checks on ownership, role or route. Only the login and rate limit questions count events over time.

IX. Experimental Method

All experiments ran on one machine with a single Uvicorn worker. The server listened on `127.0.0.1:8000` and lab mode was switched on at startup. The database was seeded with the three synthetic users and four items. Each attack script performs the same steps.

1) Confirm that the target is on the allow list and that lab mode is on.

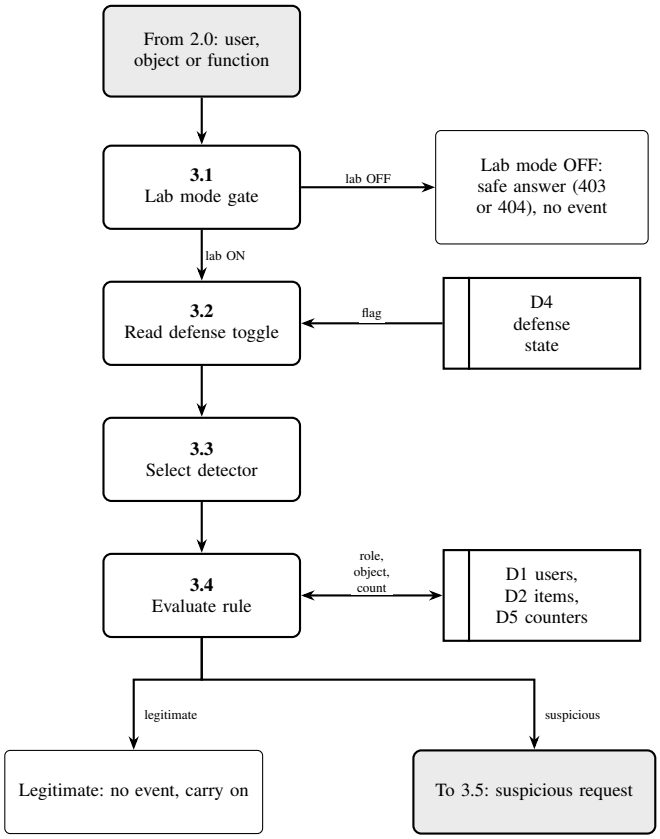


Fig. 8. DFD Level 3 (a), process 3.0, steps 3.1 to 3.4: check the request.

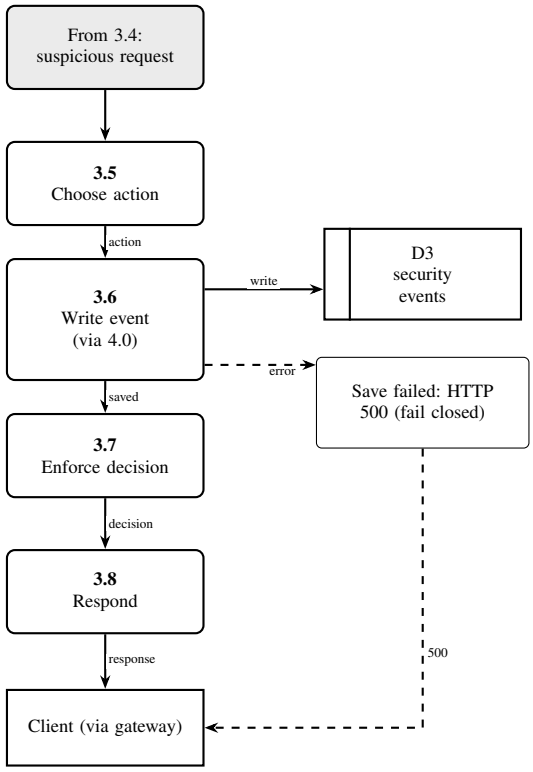


Fig. 9. DFD Level 3 (b), process 3.0, steps 3.5 to 3.8: decide, record and respond.

2) Make sure the relevant defense is off, run the attack, record the status codes and look for the matching event in the log.
3) Switch the defense on through the admin API, run the same attack again and record the same things.
4) Restore the defense switch and write a JSON file holding both runs.

I measured the HTTP status codes, whether the attack succeeded, whether the request was blocked and whether the expected security event (right category, rule and action) was found. For the rate limit experiment I also counted the 200 and 429 replies and took latency percentiles from the timing header. Detection counts as *verified* only when an event with the expected category, rule name and action appears in the log. A 403 alone is not accepted as proof of detection.

X. Results and Discussion

A. Saved results

Table VIII shows what the five saved result files contain. Each file comes from one run of the script on one occasion.

In all five modules the vulnerable run succeeded, the defended run was stopped and a matching event was verified in both runs. In each pair the first event has the action `detected` and the second has `blocked`. For BOLA the vulnerable run produced event 59 and the defended run produced event 60.

TABLE VIII
Results from the saved single runs

Risk	Defense OFF	Defense ON	Event found (off / on)
API1 BOLA	200, attack succeeded	403, blocked	Yes / Yes
API2 Auth	7 of 7 attempts got 401, account stayed open	5 got 401, then 2 got 429 (lockout at attempt 6)	Yes / Yes
API3 BOPLA	200, both hidden fields returned	403, no hidden field returned	Yes / Yes
API4 Rate	25 allowed, 0 limited	0 allowed, 25 limited	Yes / Yes
API5 BFLA	200	403	Yes / Yes

B. What I take from this

Detection and blocking are different things. For me the most useful result is that the log recorded the attack even with the defense off. A system that only blocks is not the same as a system that also tells you what happened.

A status code can mislead. A 403 might come from the defense or from some unrelated rule. Matching the event rule and action makes the check stronger.

The rate limit numbers need care. The defended run shows 25 requests limited and none allowed although the limit is 20 per minute. My guess is that the vulnerable run had just used up the same user's 60 second window so the counter was already full when the defended run began. The comparison

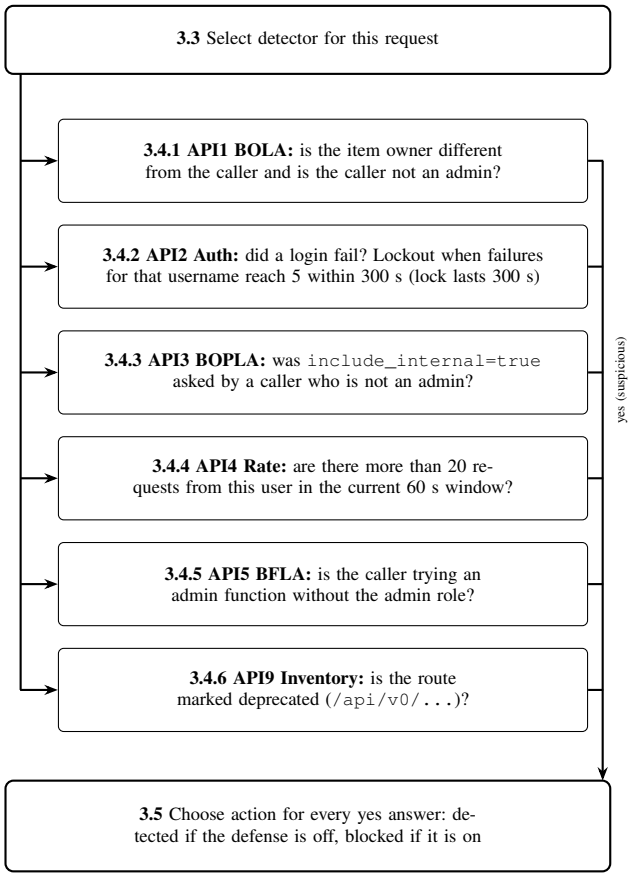


Fig. 10. DFD Level 4, process 3.4 (evaluate rule). Each rule is a yes or no question. A no answer means the request is legitimate and nothing is logged.

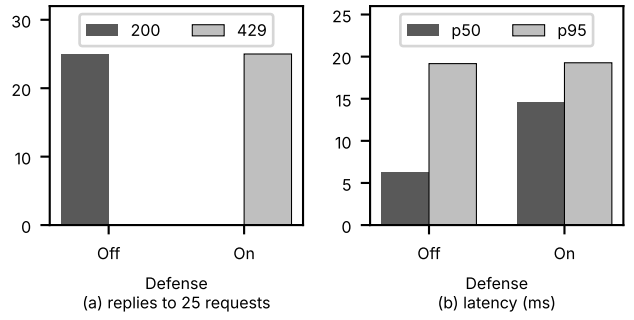


Fig. 11. Rate limit experiment from the saved run: (a) replies to the burst of 25 requests, (b) latency percentiles from the timing header.

would be cleaner if the counter were reset between the two runs. Median latency was 6.25 ms in the vulnerable run and 14.53 ms in the defended run while the 95th percentile was nearly equal (19.17 and 19.27 ms), as Fig. 11 shows. Perhaps every over-limit request writes a security event to the database and that costs time but I did not test it and one run is too little to call it overhead.

Some detector behaviour is rougher than the documentation suggests. While reading the code I saw that the authentication detector writes an event for every failed login and not only once the threshold is reached. With the defense on it labels those events `blocked` even for the first five attempts, which still received 401. The lockout works as shown in the experiment but the event meaning for API2 could be more precise. I also noticed that the hand written route list from `/admin/inventory` contains two entries (`PATCH /admin/defenses/{name}` and `POST /admin/lab-mode`) that do not match the real routes. It is a small reminder that keeping an inventory accurate is hard even in a project this size.

C. What the results do not show

The inventory module has 17 tests but no saved experiment file so I report no run for it. The evaluation script that repeats each experiment many times and computes averages exists but the summary file in the package is still a placeholder. That means I have no averages, no standard deviation and no false positive rate from live runs. The tests do check that owners and admins generate no events, which gives some comfort about false positives, but a test is not a measured rate.

XI. Limitations

- **Single runs.** Each number comes from one run per module so I can say nothing about variation.
- **In memory state.** Defense flags, the lockout tracker and the rate limiter reset on a server restart and work only with one worker. A real deployment would need a shared store such as Redis.
- **SQLite and one machine.** Fine for a lab but not like a distributed production system.
- **Partial coverage.** Six of the ten OWASP categories are implemented. Sensitive business flows, server side request forgery, security misconfiguration and unsafe consumption of APIs are missing.
- **Rule based detection.** The detectors are simple rules written for known attacks. They are not anomaly based and cannot catch an attack nobody wrote a rule for.
- **Controlled attacks.** The attack scripts are small examples and not a full picture of what a real attacker would try.
- **One example per category.** BFLA for instance is tested on a single admin endpoint and BOPLA covers only the read side.

XII. Future Work

My first priority is to run the evaluation script ten or more times per module with a reset in between so I can report means, spread and false positive counts. Next I want to add the missing OWASP categories, starting with security misconfiguration and server side request forgery against a local mock service. Moving the counters and flags to Redis would remove the single process limit and the security events could be exported to a log analysis tool. Two ideas from the literature also look worth trying: using an LLM to guess which parameters are object references [19], [21] and adding the

write side of property problems from the mass assignment work [22]. Finally I would like to run a standard scanner such as OWASP ZAP against the same server and compare its findings with what APIShield's own detectors log.

XIII. Conclusion

APIShield has a simple goal. Take one API weakness at a time, attack it, watch it get detected, switch on the fix, attack again and read the proof. For five OWASP categories I have saved runs where the attack worked with the defense off and failed with it on, with a matching event in the log each time. The sixth category is implemented and tested. The literature shows that API attacks keep growing and that access control flaws remain hard for automated tools to find. A teaching lab will not fix that. It did help me see what each flaw looks like in code, what a useful detector should record and why a status code alone is weak evidence. The results come from single runs on one machine and I have tried to say so wherever it matters.

References

[1] OWASP Foundation, "OWASP API Security Top 10 – 2023," 2023. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x00-header/

[2] Infosecurity Magazine, "Average number of daily API attacks up 113% annually" (summary of the Akamai State of the Internet report), 2026. [Online]. Available: https://infosecurity-magazine.com/news/average-number-daily-api-attacks

[3] Traceable AI, "2025 Global State of API Security," Oct. 2024. [Online]. Available: https://traceable.ai/2025-state-of-api-security

[4] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999.

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[6] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token best current practices," IETF RFC 8725, Feb. 2020.

[7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[8] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, Jun. 2017.

[9] M. Nottingham and R. Fielding, "Additional HTTP status codes," IETF RFC 6585, Apr. 2013.

[10] Salt Security, "State of API Security Report Q1 2025," Feb. 2025. [Online]. Available: https://content.salt.security/state-api-report

[11] A. Golmohammadi, M. Zhang, and A. Arcuri, "Testing RESTful APIs: A survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, pp. 27:1–27:41, Nov. 2023, doi: 10.1145/3617175.

[12] M. Zhang and A. Arcuri, "Open problems in fuzzing RESTful APIs: A comparison of tools," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, 2023, doi: 10.1145/3597205.

[13] M. Kim, Q. Xin, S. Sinha, and A. Orso, "Automated test generation for REST APIs: No time to rest yet," in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2022.

[14] E. Viglianisi, M. Dallago, and M. Ceccato, "RESTTESTGEN: Automated black-box testing of RESTful APIs," in *Proc. IEEE 13th Int. Conf. Softw. Testing, Validation and Verification (ICST)*, 2020.

[15] F. Tanveer, F. Iradat, W. Iqbal, and A. Ahmad, "Towards secure APIs: A survey on RESTful API vulnerability detection," *Comput., Mater. Continua*, vol. 84, no. 3, pp. 4223–4258, 2025, doi: 10.32604/cmc.2025.067536.

[16] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API fuzzing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, 2019, pp. 748–758, doi: 10.1109/ICSE.2019.00083.

[17] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Checking security properties of cloud service REST APIs," in *Proc. IEEE 13th Int. Conf. Softw. Testing, Validation and Verification (ICST)*, 2020, pp. 387–397, doi: 10.1109/ICST46399.2020.00046.

[18] W. Du *et al.*, "Vulnerability-oriented testing for RESTful APIs," in *Proc. 33rd USENIX Security Symp.*, 2024, pp. 739–755.

[19] I P. A. Dharmaadi, M. Alhanahnah, V.-T. Pham, F. Mohsen, and F. Turkmen, "BACFuzz: Exposing the silence on broken access control vulnerabilities in web applications," arXiv:2507.15984, 2025.

[20] Y. Huang *et al.*, "Detecting broken object-level authorization vulnerabilities in database-backed applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2024, pp. 2934–2948, doi: 10.1145/3658644.3690227.

[21] Palo Alto Networks Unit 42, "Harnessing LLMs for automating BOLA detection," 2024. [Online]. Available: https://unit42.paloaltonetworks.com/automated-bola-detection-and-ai/

[22] D. Corradini, M. Pasqua, and M. Ceccato, "Automated black-box testing of mass assignment vulnerabilities in RESTful APIs," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 2553–2564, doi: 10.1109/ICSE48619.2023.00213.

[23] L. Pan, S. Cohney, T. Murray, and V.-T. Pham, "EDEFuzz: A web API fuzzer for excessive data exposures," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024.

[24] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com